

Projet de Développement Logiciel

Groupe I1b – Université de Bordeaux

Introduction

Ce document décrit les fonctionnalités requises pour le développement d'une application Full-Stack permettant de stocker et de parcourir des images, de gérer l'authentification des utilisateurs et de classer les images à l'aide de l'intelligence artificielle (IA).

- **Stack:** Typescript, Vue.js, Java Spring Boot (Images server), postgresQL (Storing images indexes), Node.js (User validation sever), Firebase (User authentication & Cloud storage), Tailwind (CSS), Resend (Emails sending), Python/TensorFlow (Model for image classification).

Formats d'images acceptés par le serveur:

— JPEG

— PNG

Images App

1. Images server

Besoin 1 : Téléversement & indexation d'images

Description: Les images envoyées via le frontend (par des utilisateurs authentifiés) seront transmises au serveur Spring Boot via la méthode POST pour être indexées. Une copie de chaque image sera également envoyée dans le stockage cloud Firebase. Les images indexées dans la base de données seront représentées par des objets contenant les **16 champs suivants**:

- **id:** identifiant unique de l'image
- **width:** largeur originale de l'image
- **height:** hauteur originale de l'image
- **name:** nom original du fichier
- **format:** format de l'image (PNG ou JPG)
- **histogram_of_visual_words:** tableau de taille 20 représentant le nombre de caractéristiques détectées correspondant au dictionnaire visuel (voir Besoin 2)
- **histogram_2d:** histogramme HSV
- **histogram_3d:** histogramme RGB
- **img_class:** classe attribuée à l'image selon les étiquettes du dataset CIFAR-10 (voir Besoin 3)
- **img_class_confidence:** taux de confiance (en pourcentage) de la classe prédite par le modèle d'IA
- **img_url:** URL de l'image générée par le stockage cloud Firebase (voir Besoin 4)

- **img_upload_date:** timestamp indiquant la date et l'heure de l'envoi
- **author:** identifiant de l'utilisateur (Firebase) ayant envoyé l'image
- **like_count:** nombre de mentions "J'aime" reçues par l'image
- **download_count:** nombre de fois que l'image a été téléchargée
- **title:** titre de l'image (texte)

Besoin 2 : Détection améliorée de similarité

Description: En plus des descripteurs RGB et HSV déjà utilisés pour la détection de similarité, des images similaires peuvent être identifiées en analysant la **distance entre leurs histogrammes de mots visuels**. Ces histogrammes sont calculés à partir d'un **dictionnaire visuel** composé de **10 clusters**.

Ces clusters représentent des motifs visuels généraux calculés à partir de la moyenne des caractéristiques détectées dans les images d'entraînement. Pour leur calcul, on utilise le **jeu de données CIFAR-10**.

Besoin 3: AI Model for classifying images

Description: Les images soumises au serveur doivent d'abord être classifiées selon **10 catégories** correspondant aux classes du **jeu de données CIFAR-10**. Pour cela, un modèle d'intelligence artificielle est construit avec **TensorFlow** et **Python**, en suivant une architecture **CNN (réseau de neurones convolutifs)** composée de **6 couches profondes**.

Ce modèle doit normalement atteindre une précision de **86 %** sur les données d'entraînement et de test.

Remarque :

Les images envoyées par les utilisateurs doivent d'abord être **redimensionnées à 32×32 pixels** afin de correspondre au format d'entrée attendu par le modèle.

Besoin 4: Stockage des images dans le cloud

Description: Lorsqu'une image est soumise au serveur, la requête doit fournir une **URL publique** permettant d'afficher cette image côté frontend. Pour cela, le projet utilise **Firebase Cloud Storage** comme système de stockage.

Une fois la fonctionnalité de stockage activée dans le projet Firebase (voir tutoriel :

👉 <https://firebase.google.com/docs/web/setup?hl=fr>), le **frontend** peut téléverser les fichiers image dans le cloud et récupérer l'URL correspondante.

Remarque :

Toutes les requêtes vers le projet Firebase peuvent être effectuées côté frontend, y compris la génération de l'URL de l'image. Cependant, cette URL devra être transmise au serveur (Spring Boot) en tant que **métadonnée associée à l'image**.

Besoin 5 : Gestion des likes (côté backend)

Le backend doit fournir **deux routes sécurisées** permettant de gérer le nombre de likes d'une image :

- Une **route pour incrémenter** le compteur de likes lorsqu'un utilisateur **aime** une image.
- Une **route pour décrémenter** le compteur lorsqu'un utilisateur **retire son like**.
-

Besoin 7 : Incrémentation des téléchargements d'image

Chaque fois qu'un utilisateur télécharge une image depuis l'interface, le backend doit **incrémenter le compteur de téléchargements** (download_count) associé à cette image.

Détails techniques :

- L'identifiant de l'image (id) doit être transmis à la route backend,
- Le compteur doit être mis à jour dans la base de données (PostgreSQL),
- Aucune authentification n'est requise ici : le compteur peut être incrémenté pour tous les utilisateurs (y compris visiteurs anonymes).

Souhaites-tu que je t'écrive les endpoints REST correspondants (POST /like, POST /unlike, etc.) ou que je t'aide à implémenter les routes directement en Node.js ou Spring Boot ?

40

2 . **Firestore:**

Besoin 1 : Système d'authentification avec Firebase

Description :

L'application propose un système d'authentification permettant aux utilisateurs de se connecter. Cette étape est essentielle, car seuls les utilisateurs authentifiés peuvent soumettre des images et effectuer diverses actions (liker des images, créer des albums, etc.).

Comme mentionné précédemment, la création d'un projet Firebase est nécessaire non seulement pour le téléversement d'images dans le cloud, mais aussi pour la gestion de l'authentification des utilisateurs.

Configuration de Firebase :

Pour installer Firebase dans le projet, exécute la commande suivante dans le répertoire du frontend :

```
npm install firebase
```

Crée ensuite un dossier firebase dans le frontend qui contiendra le système d'authentification ainsi qu'un fichier appelé firebase.ts. Ce fichier stockera les informations d'identification nécessaires pour accéder:

- au stockage cloud,
- à la base de données Firebase,
- au service d'authentification.

Ce fichier doit ressembler à ceci :

```

1  import { configurations } from "../app-configuration";
2  import { initializeApp } from "firebase/app";
3  import { getAuth } from "firebase/auth";
4  import { getFirestore } from "firebase/firestore";
5  import { getStorage } from "firebase/storage";
6
7  const firebaseConfig = {
8    apiKey: "AIzaSyCqBGiU6pklEiOL84qLIOu3dXrKg7dKJhE",
9    authDomain: "images-app-6bbe5.firebaseio.com",
10   projectId: "images-app-6bbe5",
11   storageBucket: "images-app-6bbe5.firebaseio.com",
12   messagingSenderId: "440600748418",
13   appId: "1:440600748418:web:34b49c1f042e07c7f1ef9b"
14 };
15
16 export const actionCodeSettings = {
17   url: `${configurations.host}?verified=true`, // URL where the link will redirect
18   handleCodeInApp: true, // This must be true for redirects to work on mobile
19 };
20 // Initialize Firebase
21 const app = initializeApp(firebaseConfig);
22 const auth=getAuth(app)
23 const db = getFirestore(app)
24 const storage=getStorage(app)
25
26 export {app,auth,db,storage}

```

Remarque : L'objet `firebaseConfig` peut être obtenu directement depuis la console du projet Firebase.

Authentification

Le système d'authentification doit permettre aux utilisateurs de créer un compte et de se connecter. Pour se connecter, deux options sont disponibles :

- via leur compte Google
- ou en renseignant une adresse e-mail et un mot de passe

Pour s'inscrire avec une adresse e-mail et un mot de passe, les utilisateurs doivent d'abord vérifier leur adresse e-mail via un lien qui leur sera envoyé. Ce lien contiendra un token JWT qui expire au bout d'une heure. L'utilisateur devra donc cliquer sur ce lien dans ce délai pour activer son compte.

Une fois l'e-mail vérifié, l'utilisateur pourra se connecter et réinitialiser son mot de passe à tout moment. Pour réinitialiser un mot de passe, un lien contenant un token JWT sécurisé lui sera envoyé par e-mail. En cliquant sur ce lien, l'utilisateur sera redirigé vers une page où il pourra modifier son mot de passe en toute sécurité.

Envoi du lien de vérification / réinitialisation du mot de passe

Ces deux types de liens doivent être envoyés à l'adresse e-mail de l'utilisateur à l'aide d'un service appelé **Resend**, en utilisant un **domaine e-mail personnalisé**. Pour utiliser Resend, une **clé API** est nécessaire afin d'envoyer les e-mails.

L'envoi des e-mails doit se faire côté backend, c'est pourquoi un **serveur Node.js** sera mis en place dans le projet. Celui-ci se chargera de :

- générer les **tokens JWT** pour renforcer la sécurité,
- stocker ces tokens dans les documents utilisateur de la base Firebase **avant** la vérification de l'e-mail ou la réinitialisation du mot de passe.

Remarque : *Firebase gère automatiquement les sessions utilisateur et la connexion/déconnexion. Le mécanisme d'authentification repose également sur les **tokens JWT**.*

Stockage des utilisateurs

Les données des utilisateurs authentifiés sont stockées dans la base sous forme d'objets comportant **11 champs** :

- id : identifiant unique de l'utilisateur
- userName : nom choisi par l'utilisateur
- email : adresse e-mail
- favImages : liste des images mises en favoris
- uploadedImages : images téléversées
- viewedImages : images consultées
- albums : galeries d'images créées par l'utilisateur
- joinDate : date d'inscription
- emailVerified : initialisé à false, devient true une fois l'e-mail vérifié
- photoUrl : image de profil de l'utilisateur

Fonctionnalités Utilisateur

Besoin 1 : Téléversement d'images

Les utilisateurs authentifiés peuvent téléverser des images sur le site et les supprimer à tout moment s'ils le souhaitent.

Besoin 2 : Ajout aux favoris

Les utilisateurs peuvent ajouter des images à leur liste de favoris et les retirer à tout moment. Ces images doivent être stockées uniquement sous forme d'identifiants (id), afin de faire référence aux images réelles côté backend sans dupliquer les données.

Besoin 3 : Création de galeries d'images

Les utilisateurs peuvent créer des galeries d'images personnalisées. Chaque galerie peut :

- avoir un **nom**,
- être **supprimée**,
- permettre l'**ajout** ou la **suppression** d'images à tout moment.

Les images présentes dans une galerie doivent être consultables via un **carrousel interactif**.

Besoin 4 : Historique des images consultées

Chaque fois qu'un utilisateur consulte une image, son **historique de visualisation** est automatiquement mis à jour pour refléter cette action.

Besoin 5 : Profil utilisateur / Page personnelle

Chaque utilisateur dispose d'une page de profil personnalisée lui permettant de :

- consulter ses images téléversées,
- consulter ses images favorites,
- accéder à ses galeries,
- consulter son historique de visualisation,
- **se déconnecter**,
- **modifier son nom d'utilisateur**,
- **changer sa photo de profil**,
- **supprimer définitivement son compte**.

En complément, chaque utilisateur dispose également d'une **page publique** visible par tous les visiteurs du site, affichant uniquement les images qu'il a téléversées.

Interface & Design

Besoin 1 : Affichage des images (Images UI)

Chaque image dans l'application peut être affichée **sous forme de carte** (card) ou **en pleine page**.

Lors de l'affichage d'une image, l'interface doit afficher les informations suivantes :

- Le **compteur de likes**,
- Le **nombre de téléchargements**,
- Le **nom de l'auteur** et son **image de profil**.

Lorsque l'on clique sur le **nom de l'auteur** ou sur son image de profil, l'utilisateur est redirigé vers la **page publique** de l'auteur.

Enfin, lorsque l'image est consultée **en mode plein écran**, une **section dédiée aux images similaires** doit apparaître, présentant les images les plus proches visuellement (selon les 3 descripteurs implémentées).

Besoin 2 : Design responsive & Thème unifié avec Tailwind CSS

L'interface de l'application doit être construite avec **Tailwind CSS**, en tirant parti de ses classes utilitaires pour garantir une **mise en page cohérente, moderne et maintenable**.

Exigences :

- L'ensemble de l'interface doit être **responsive**, avec une expérience fluide sur ordinateurs, tablettes et smartphones.
- L'**apparence visuelle de l'application** doit suivre un **thème graphique unifié**, avec une **palette de couleurs cohérente** sur toutes les pages.
- La **couleur principale** (accent) du thème est définie comme étant #5ebf88 (un vert doux), et devra être utilisée pour :
 - Les boutons principaux,
 - Les icônes actives,
 - barre de navigation.

Adresse publique du projet: <https://images-app-production.up.railway.app/signup>

